UNIVERSIDAD ARGENTINA DE LA EMPRESA
FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

TRABAJO PRÁCTICO Nº1
"TDAs: Conceptos Básicos "

Integrantes

Apellido, Nombre	*Legajo*
Gueler Martin	*1220964*
Constan Langer, Maria Guadalupe	*1144964*

Profesor

COEDO, MAXIMILIANO GONZALO

Curso

Lunes - Noche

ENUNCIADO

Definición, implementación y utilización

En este trabajo van a definir, implementar y utilizar tres TDAs clásicos: Pila, Cola y Cola con Prioridad, usando implementaciones estáticas (basadas en arreglos). El objetivo es consolidar la diferencia entre especificación (qué hace el TDA) e implementación (cómo lo hace), y practicar la resolución de problemas usando exclusivamente las operaciones de la interfaz, sin asumir ningún detalle interno de la implementación.

Condiciones de entrega Trabajo grupal. Lenguaje: Java. Cada método pedido debe resolverse usando únicamente las operaciones del TDA correspondiente (crear, apilar/desapilar, encolar/desencolar, insertar/extraer, esVacia, etc.) — no accedan directamente al arreglo interno ni al índice de tope/frente desde fuera de la implementación. Se evalúa tanto la corrección como la complejidad temporal de cada solución.

Parte 1 — TDA Pila

1.1 Especificación

Escriban la especificación del TDA Pila siguiendo el formato visto en clase: dominio y operaciones, cada una con su signatura y sus condiciones de precondición/postcondición. Como mínimo debe incluir: crear, apilar, desapilar, tope, esVacia.

1.2 Implementación

Implementen el TDA Pila de forma estática (con un arreglo de tamaño fijo) de dos maneras distintas:

● Variante A: el tope de la pila se mantiene en el último lugar ocupado del arreglo.

● Variante B: el tope de la pila se mantiene siempre en la posición 0 del arreglo.

Para cada variante, indiquen la complejidad temporal de apilar, desapilar y tope. Con esa información, respondan y justifiquen: ¿cuál de las dos variantes conviene usar? ¿Depende de algo?

1.3 Utilización

Usando únicamente las operaciones del TDA Pila (sin importar cuál de las dos variantes uses — el código debe funcionar igual con cualquier implementación), resuelvan:

1. pasarPila(origen: Pila): Pila — devuelve una nueva pila con todos los elementos de origen, dejando origen vacía.

2. copiarPila(p: Pila): Pila — devuelve una copia de p, dejando p en su estado original (mismo contenido y mismo orden que antes de llamar al método).

3. invertirPila(p: Pila): Pila — devuelve una nueva pila con los elementos de p en el orden inverso. Debe resolverse en forma recursiva.

4. masDeUnaOcurrencia(p: Pila): boolean — indica si algún elemento aparece más de una vez en p. Al finalizar, p debe quedar con su contenido y orden original.

5. eliminarImpares(p: Pila): Pila — devuelve una nueva pila con los elementos de p que son pares, en el mismo orden relativo en que estaban en p.

Parte 2 — TDA Cola

2.1 Especificación

Escriban la especificación del TDA Cola: dominio y operaciones (crear, encolar, desencolar, frente, esVacia), con sus pre/postcondiciones.

2.2 Implementación

Implementen el TDA Cola de forma estática (con un arreglo de tamaño fijo) de dos maneras distintas:

● Variante A — Cola lineal simple: frente y fin avanzan siempre hacia adelante en el arreglo (sin reutilizar los lugares que quedan libres al desencolar).

● Variante B — Cola circular: frente y fin avanzan usando aritmética modular, reutilizando los lugares liberados.

Analicen la complejidad temporal de encolar y desencolar en cada variante, y justifiquen cuál conviene usar en la práctica.

2.3 Utilización

Usando únicamente las operaciones del TDA Cola, resuelvan:

6. pasarCola(origen: Cola): Cola — devuelve una nueva cola con todos los elementos de origen, en el mismo orden, dejando origen vacía.

7. invertirColaConPila(c: Cola): Cola — devuelve una nueva cola con los elementos de c en orden inverso, usando una Pila auxiliar.

8. invertirColaSinPila(c: Cola): Cola — resuelve el mismo problema que el punto anterior, pero sin usar ninguna estructura auxiliar (ni Pila, ni arreglo, ni Cola extra): la inversión debe lograrse únicamente con recursión pura sobre la propia Cola.

9. finalCoincide(c1: Cola, c2: Cola, k: entero): boolean — indica si los últimos k elementos de c1 coinciden, en el mismo orden, con los últimos k elementos de c2. Ambas colas deben quedar con su contenido y orden original al finalizar.

Parte 3 — TDA Cola con Prioridad

3.1 Especificación

Escriban la especificación del TDA Cola con Prioridad: dominio y operaciones (crear, insertar — que recibe un elemento y una prioridad —, extraerMax, verMax, esVacia), con sus pre/postcondiciones. Definan también qué hace su TDA cuando dos elementos tienen la misma prioridad (criterio de desempate).

3.2 Implementación

Implementen el TDA Cola con Prioridad de forma estática (con un arreglo de tamaño fijo) de dos maneras distintas:

● Variante A — Desordenada: los elementos se insertan en el primer lugar libre, sin mantener ningún orden; extraerMax debe recorrer el arreglo para encontrar el máximo.

● Variante B — Ordenada: los elementos se mantienen siempre ordenados por prioridad dentro del arreglo, de forma que extraerMax accede directamente al máximo.

Para cada variante, dado que ambas deben respetar el criterio de desempate definido en 3.1, analicen la complejidad temporal de insertar, extraerMax y verMax. Con esos datos, discutan: ¿existe una variante que sea mejor que la otra en todos los casos, o la elección depende del contexto de uso? Justifiquen.

3.3 Utilización

Usando únicamente las operaciones del TDA Cola con Prioridad, resuelvan:

10. combinar(cp1: ColaPrioridad, cp2: ColaPrioridad): ColaPrioridad — devuelve una nueva cola con prioridad que contiene todos los elementos de cp1 y cp2, respetando el criterio de desempate definido en 3.1 (es decir, el orden relativo de inserción entre elementos de igual prioridad debe conservarse). Ambas colas de entrada deben quedar vacías al finalizar.

11. invertirColaConColaPrioridad(c: Cola): Cola — devuelve una nueva cola con los elementos de c en orden inverso, usando una ColaPrioridad auxiliar (pensar qué prioridad asignarle a cada elemento a medida que se recorre c).

12. sumarValoresPrioridadPar(cp: ColaPrioridad): entero — devuelve la suma de los valores de todos los elementos cuya prioridad es un número par. cp debe quedar con su contenido y orden original al finalizar.

Entrega

Entreguen el código fuente completo (una clase por TDA y por variante, más una clase con los métodos de utilización de cada parte) junto con un breve informe (puede ser el mismo Word/PDF con las respuestas a las consignas de análisis) que incluya:

● Las especificaciones pedidas en 1.1, 2.1 y 3.1.

● Las tablas de complejidad temporal y las justificaciones pedidas en 1.2, 2.2 y 3.2.

● Para cada método de las secciones de utilización, la complejidad temporal de la solución pr

1.1 TDA PILA

```
Operaciones:

crear() → Pila

    post: devuelve una pila vacía.

apilar(p: Pila, x: elemento) → Pila

    pre: p no está llena.

    post: x queda en el tope de p.

desapilar(p: Pila) → Pila

    pre: p no está vacía.

    post: se elimina el elemento del tope de p.

tope(p: Pila) → elemento

    pre: p no está vacía.

    post: devuelve el elemento del tope sin eliminarlo.

esVacia(p: Pila) → boolean

    post: devuelve true si p no tiene elementos y false en caso contrario.

esLlena(p: Pila) → boolean

    post: devuelve true si p alcanzó su capacidad máxima y false en caso contrario.
```

1.2

El TDA se implementa de forma estática mediante un arreglo de tamaño fijo. Se presentan dos variantes diferenciadas por la posición en la que se mantiene el tope de la pila.

VARIANTE A: tope en el último lugar ocupado.

En esta implementación, los elementos se almacenan desde la posición 0 del arreglo y el tope se encuentra siempre en la última posición ocupada.

Se utiliza una variable cantidad para indicar la cantidad de elementos almacenados.

```
    private int[] datos;
    private int cantidad;
    private int capacidad;

    public PilaEstaticaB(int capacidad) {
        this.capacidad = capacidad;
        this.datos = new int[capacidad];
        this.cantidad = 0;
    }
    public boolean esVacia() {
        return cantidad == 0;
    }
    public boolean esLlena() {
        return cantidad == capacidad;
    }
    public void apilar(int x) {
        if (!esLlena()) {
            for (int i = cantidad; i > 0; i--) {
                datos[i] = datos[i - 1];
            }
            datos[0] = x;
            cantidad++;
        }
    }
    public int desapilar() {
        if (!esVacia()) {
            int resultado = datos[0];

            for (int i = 0; i < cantidad - 1; i++) {
                datos[i] = datos[i + 1];
            }
            cantidad--;
            return resultado;
        }
        return -1;
    }
    public int tope() {
        if (!esVacia()) {
            return datos[0];
        }
        return -1;
    }
}
```

Rango de complejidad

apilar	O(1)
desapilar	O(1)
tope	O(1)

Todas las operaciones se realizan directamente sobre una posición del arreglo, sin necesidad de desplazar elementos

En esta implementación, el tope de la pila se mantiene siempre en la posición 0 del arreglo. Al agregar un elemento, es necesario desplazar todos los elementos existentes una posición hacia la derecha para dejar libre la posición 0.

Al eliminar el tope, los elementos restantes deben desplazarse una posicion hacia la izquierda.

```
Public class PilaEstaticaB implements Pila {

    private int[] datos;
    private int cantidad;
    private int capacidad;

    public PilaEstaticaB(int capacidad) {
        this.capacidad = capacidad;
        this.datos = new int[capacidad];
        this.cantidad = 0;
    }
    public boolean esVacia() {
        return cantidad == 0;
    }
    public boolean esLlena() {
        return cantidad == capacidad;
    }
    public void apilar(int x) {
        if (!esLlena()) {
            for (int i = cantidad; i > 0; i--) {
                datos[i] = datos[i - 1];
            }
            datos[0] = x;
            cantidad++;
        }
    }
    public int desapilar() {
        if (!esVacia()) {
            int resultado = datos[0];

            for (int i = 0; i < cantidad - 1; i++) {
                datos[i] = datos[i + 1];
            }
            cantidad--;
            return resultado;
        }
        return -1;
    }
    public int tope() {
        if (!esVacia()) {
            return datos[0];
        }
        return -1;
    }
}
```

apilar	O(n)
desapilar	O(n)
tope	O(1)

En apilar y desapilar es necesario desplazar elementos del arreglo, por lo que el tiempo depende de la cantidad de elementos almacenados.

¿Cuál variante conviene utilizar?

En términos generales, **conviene utilizar la Variante A**, ya que permite realizar apilar, desapilar y tope en tiempo constante, es decir, **O(1)**.
La Variante B requiere desplazar elementos del arreglo cada vez que se agrega o elimina un elemento, por lo que apilar y desapilar tienen complejidad **O(n)**.

Por lo tanto, si no existe otra restricción, la **Variante A es más eficiente**.

La elección podría depender del contexto de utilización de la estructura. Sin embargo, para una Pila, donde las operaciones principales son apilar y desapilar, resulta más conveniente mantener el tope en el último lugar ocupado, porque permite realizar ambas operaciones sin desplazamientos.

1.3 Utilización

Las siguientes operaciones utilizan únicamente las operaciones definidas por la interfaz del TDA Pila, sin acceder directamente al arreglo interno ni a la posición del tope.

1. pasarPila

```
public static Pila pasarPila(Pila origen) {
    Pila auxiliar = new PilaEstaticaA();
    Pila resultado = new PilaEstaticaA();

    while (!origen.esVacia()) {
        auxiliar.apilar(origen.desapilar());
    }

    while (!auxiliar.esVacia()) {
        resultado.apilar(auxiliar.desapilar());
    }

    return resultado;
}
```

Complejidad temporal: O(n)

2. copiarPila

```
public static Pila copiarPila(Pila p) {
    Pila auxiliar = new PilaEstaticaA();
    Pila resultado = new PilaEstaticaA();

    while (!p.esVacia()) {
        auxiliar.apilar(p.desapilar());
    }

    while (!auxiliar.esVacia()) {
        int elemento = auxiliar.desapilar();
        p.apilar(elemento);
        resultado.apilar(elemento);
    }

    return resultado;
}
```

Complejidad temporal: O(n)

La pila original queda exactamente con el mismo contenido y orden.

3. invertirPila

```
public static Pila invertirPila(Pila p) {
    Pila copia = copiarPila(p);
    Pila resultado = new PilaEstaticaA();

    invertirRecursivo(copia, resultado);

    return resultado;
}

private static void invertirRecursivo(Pila origen, Pila resultado) {
    if (!origen.esVacia()) {
        int elemento = origen.desapilar();
        resultado.apilar(elemento);
        invertirRecursivo(origen, resultado);
    }
}
```

Complejidad temporal: O(n)

La operación se resuelve de forma recursiva y la pila original no se modifica.

4. masDeUnaOcurrencia

```
public static boolean masDeUnaOcurrencia(Pila p) {
    Pila copia = copiarPila(p);

    while (!copia.esVacia()) {
        int elemento = copia.desapilar();
        Pila auxiliar = new PilaEstaticaA();
        boolean encontrado = false;

        while (!copia.esVacia()) {
            int actual = copia.desapilar();

            if (actual == elemento) {
                encontrado = true;
            }

            auxiliar.apilar(actual);
        }

        while (!auxiliar.esVacia()) {
            copia.apilar(auxiliar.desapilar());
        }

        if (encontrado) {
            return true;
        }
    }

    return false;
}
```

Complejidad temporal: $O(n^2)$

La pila original p conserva su contenido y su orden.

5. eliminarImpares

```
public static Pila eliminarImpares(Pila p) {
    Pila copia = copiarPila(p);
    Pila auxiliar = new PilaEstaticaA();
    Pila resultado = new PilaEstaticaA();

    while (!copia.esVacia()) {
        int elemento = copia.desapilar();

        if (elemento % 2 == 0) {
            auxiliar.apilar(elemento);
        }
    }

    while (!auxiliar.esVacia()) {
        resultado.apilar(auxiliar.desapilar());
    }

    return resultado;
}
```

Complejidad temporal: O(n)

La pila original p no se modifica y la nueva pila contiene únicamente los elementos pares, manteniendo el mismo orden relativo que tenían en p.

Parte 2 — TDA Cola

2.1 Especificación

TDA Cola

Dominio: Cola = secuencia finita de elementos que respeta el principio FIFO (*First In, First Out*), es decir, el primer elemento en ingresar es el primero en salir.

```
Operaciones:
crear() → Cola
Post: devuelve una cola vacía.
encolar(c: Cola, x: elemento) → Cola
Pre: c no está llena.
Post: x queda ubicado al final de c.
desencolar(c: Cola) → elemento
Pre: c no está vacía.
Post: se elimina y devuelve el elemento que se encontraba al frente de c.
frente(c: Cola) → elemento
Pre: c no está vacía.
Post: devuelve el elemento que se encuentra al frente de c sin eliminarlo.
esVacia(c: Cola) → boolean
Post: devuelve true si c no tiene elementos y false en caso contrario.
esLlena(c: Cola) → boolean
Post: devuelve true si c alcanzó su capacidad máxima y false en caso contrario.
```

2.2 Implementación

La Cola se implementa mediante un arreglo de tamaño fijo. Se utilizan dos índices:

- **frente:** indica la posición del primer elemento de la cola.
- **fin:** indica la posición donde se encuentra o se agregará el último elemento.

Variante A — Cola lineal simple

En esta variante, tanto frente como fin avanzan siempre hacia adelante en el arreglo. Cuando se desencola un elemento, el espacio que queda libre no se reutiliza.

Por ejemplo:

La principal desventaja es que, aunque existan posiciones libres al comienzo del arreglo, no pueden volver a utilizarse. Cuando fin llega al final del arreglo, la cola se considera llena para nuevos elementos.

Complejidad temporal:

encolar	O(1)
desencolar	O(1)
frente	O(1)

Implementación conceptual:

```
public class ColaEstaticaLineal {
    private int[ ] datos;
    private int frente;
    private int fin;

    public ColaEstaticaLineal(int capacidad) {
        datos = new int[capacidad];
        frente = 0;
        fin = 0;
    }

    public boolean esVacia() {
        return frente == fin;
    }

    public void encolar(int elemento) {
        datos[fin] = elemento;
        fin++;
    }

    public int desencolar() {
        int elemento = datos[frente];
        frente++;
        return elemento;
    }

    public int frente() {
        return datos[frente];
    }
}
```

Variante B — Cola circular

En esta variante, el arreglo se considera circular. Cuando frente o fin llegan al final del arreglo, vuelven al comienzo utilizando **aritmética modular**.

De esta manera, las posiciones que quedan libres después de desencolar pueden reutilizarse.

Complejidad temporal:

encolar	O(1)
desencolar	O(1)
frente	O(1)

Implementación conceptual:

```
public class ColaEstaticaCircular {
    private int[] datos;
    private int frente;
    private int fin;
    private int cantidad;

    public ColaEstaticaCircular(int capacidad) {
        datos = new int[capacidad];
        frente = 0;
        fin = 0;
        cantidad = 0;
    }

    public boolean esVacia() {
        return cantidad == 0;
    }

    public boolean esLlena() {
        return cantidad == datos.length;
    }

    public void encolar(int elemento) {
        if (!esLlena()) {
            datos[fin] = elemento;
            fin = (fin + 1) % datos.length;
            cantidad++;
        }
    }

    public int desencolar() {
        if (!esVacia()) {
            int elemento = datos[frente];
            frente = (frente + 1) % datos.length;
            cantidad--;
            return elemento;
        }
        return -1;
    }

    public int frente() {
        return datos[frente];
    }
}
```

Comparación y conclusión

Ambas variantes tienen O(1) en encolar y desencolar. Sin embargo, la cola circular conviene utilizar en la práctica, porque permite reutilizar las posiciones del arreglo que quedan libres después de desencolar.

La cola lineal es más sencilla de implementar, pero desperdicia espacio: una vez que fin llega al final del arreglo, no puede utilizar las posiciones que quedaron libres al principio.

Por lo tanto, la cola circular es más eficiente en el uso de la memoria y aprovecha mejor la capacidad disponible del arreglo, manteniendo además las operaciones principales en O(1).

Parte 2 -TDA Cola

2.2 Implementación

La Cola se implementa mediante un arreglo de tamaño fijo. Se presentan dos variantes, diferenciadas por cómo se mueven los índices dentro del arreglo.

Variante A - Cola lineal simple

Los índices frente (primer elemento) y fin (próxima posición libre) avanzan siempre hacia adelante. Cuando se desencola, el lugar que queda libre al comienzo no se reutiliza. Por eso, aunque queden posiciones libres al principio, la cola se considera llena cuando fin llega al final del arreglo.

- encolar: coloca el elemento en datos[fin] e incrementa fin.
- desencolar: devuelve datos[frente] e incrementa frente.
- frente: devuelve datos[frente].

Operación	Complejidad
encolar	O(1)
desencolar	O(1)
frente	O(1)

Todas trabajan sobre una única posición del arreglo, sin desplazar elementos. **Desventaja:** desperdicia espacio; una vez que fin alcanza el final, no se pueden usar las posiciones que quedaron libres al principio.

Variante B - Cola circular

El arreglo se considera **circular**: cuando frente o fin llegan al final, vuelven al comienzo mediante **aritmética modular** (% capacidad). Así, las posiciones liberadas al desencolar se reutilizan. Se mantiene frente y cantidad; el fin se calcula como (frente + cantidad) % capacidad.

Operación	Complejidad
encolar	O(1)
desencolar	O(1)
frente	O(1)

Comparación y conclusión - ¿cuál conviene?

Ambas variantes tienen O(1) en encolar, desencolar y frente, por lo que en tiempo son equivalentes. La diferencia está en el uso de memoria:

- La cola lineal es más simple de implementar, pero desperdicia espacio: cuando fin llega al final del arreglo, la considera llena aunque haya lugares libres al comienzo.
- La cola circular reutiliza esos lugares, aprovechando toda la capacidad del arreglo y manteniendo las operaciones en O(1).

En la práctica conviene la cola circular, porque logra el mismo costo temporal que la lineal pero aprovecha mejor la memoria disponible. La lineal solo se justifica si se busca la máxima simplicidad de código y se sabe que la cantidad total de encolados no superará la capacidad del arreglo.

2.3 Utilización

Todos los métodos usan solo operaciones de la interfaz Cola (y de Pila en el método 7). Funcionan con cualquiera de las dos variantes de implementación.

6. pasarCola(origen: Cola): Cola

Devuelve una nueva cola con todos los elementos de origen, en el mismo orden, dejando origen vacía.

```
public static <T> Cola<T> pasarCola(Cola<T> origen) {
        Cola<T> nueva = new ColaCircular<>();
        while (!origen.esVacia()) {
        nueva.encolar(origen.desencolar());
        }
        return nueva;
}
```

Como se desencola desde el frente y se encola al final, el orden se conserva. **Complejidad temporal: O(n).**

7. invertirColaConPila(c: Cola): Cola

Devuelve una nueva cola con los elementos de c en orden inverso, usando una **Pila** auxiliar.

```
public static <T> Cola<T> invertirColaConPila(Cola<T> c) {
        Pila<T> pila = new PilaEstatica<>();
        while (!c.esVacia()) {
        pila.apilar(c.desencolar());
        }
        Cola<T> resultado = new ColaCircular<>();
        while (!pila.esVacia()) {
        resultado.encolar(pila.desapilar());
        }
        return resultado;
}
```

El truco es combinar el comportamiento FIFO de la cola con el LIFO de la pila: al volcar la cola en la pila y luego la pila en una cola nueva, el orden se invierte. c queda vacía al finalizar. **Complejidad temporal: O(n).**

8. invertirColaSinPila(c: Cola): Cola

Mismo resultado que el punto 7, pero sin ninguna estructura auxiliar (ni Pila, ni arreglo, ni Cola extra): solo recursión pura sobre la propia cola.

```
public static <T> Cola<T> invertirColaSinPila(Cola<T> c) {
        if (c.esVacia()) {
        return new ColaCircular<>();
        }
        T x = c.desencolar();
        Cola<T> invertida = invertirColaSinPila(c);
        invertida.encolar(x);
        return invertida;
}
```

Idea: se saca el frente x, se invierte recursivamente el resto y recién entonces se agrega x al final. Como x era el primer elemento y se agrega último, la cola queda invertida. La única cola que existe es la que se devuelve (se crea en el caso base y se propaga hacia arriba); no se usa ninguna estructura auxiliar. c queda vacía al finalizar. Complejidad temporal: O(n), con O(n) de profundidad de recursión (pila de llamadas del sistema).

9. finalCoincide(c1: Cola, c2: Cola, k: entero): boolean

Indica si los últimos k elementos de c1 coinciden, en el mismo orden, con los últimos k de c2. **Ambas colas quedan con su contenido y orden original.**

```
public static <T> boolean finalCoincide(Cola<T> c1, Cola<T> c2, int k) {
        if (k <= 0) return true;
        Cola<T> ultimos1 = ultimosK(c1, k);
        Cola<T> ultimos2 = ultimosK(c2, k);
        if (ultimos1 == null || ultimos2 == null) return false; // menos de k elementos
        while (!ultimos1.esVacia()) {
        T a = ultimos1.desencolar();
        T b = ultimos2.desencolar();
        if (!a.equals(b)) return false;
        }
        return true;
}

// Nueva cola con los últimos k elementos de c (en orden), dejando c intacta.
// Devuelve null si c tiene menos de k elementos.
private static <T> Cola<T> ultimosK(Cola<T> c, int k) {
        Cola<T> aux = new ColaCircular<>();
        int n = 0;
        while (!c.esVacia()) { aux.encolar(c.desencolar()); n++; }   // vaciar y contar
        Cola<T> ultimos = (n >= k) ? new ColaCircular<>() : null;
        int i = 0;
        while (!aux.esVacia()) {
        T x = aux.desencolar();
        c.encolar(x);                          // restaurar c
        if (ultimos != null && i >= n - k) ultimos.encolar(x); // quedarse con los últimos k
        i++;
        }
        return ultimos;
}
```

Se vacía cada cola en una auxiliar para contar su tamaño n; al restaurarla se reencolan los elementos y se retienen solo los de posición >= n - k (los últimos k). Luego se comparan en orden. Este método sí puede usar estructuras auxiliares porque el enunciado solo exige que c1 y c2 queden con su contenido y orden original. **C**omplejidad temporal: O(n1 + n2), donde n1 y n2 son los tamaños de las colas. *Precondiciones:* k >= 0 y elementos no nulos (se comparan con equals). Si k supera el tamaño de alguna cola, devuelve false.

Parte 3 - TDA Cola con Prioridad

3.1 Especificación

Dominio: colección de pares (elemento, prioridad), donde la prioridad es un entero. A mayor valor de prioridad, mayor prioridad (el máximo es el que se extrae).

Operaciones (con precondición/postcondición):

- **crear** -> ColaPrioridad *post:* devuelve una cola con prioridad vacía.
- **insertar**(e, p) : ColaPrioridad × Elemento × Entero -> ColaPrioridad *pre:* la cola no está llena. *post:* agrega el elemento e con prioridad p.
- **extraerMax**() : ColaPrioridad -> Prioritario *pre:* la cola no está vacía. *post:* elimina y devuelve el par de mayor prioridad (aplicando el criterio de desempate).

- **verMax**() : ColaPrioridad -> Prioritario *pre:* la cola no está vacía. *post:* devuelve (sin eliminar) el par de mayor prioridad, con el mismo criterio de desempate que extraerMax.
- **esVacia**() : ColaPrioridad -> boolean *post:* true si y solo si no hay elementos.
- **esLlena**() : ColaPrioridad -> boolean — observador propio de la implementación estática.

Criterio de desempate: FIFO por orden de inserción. Si dos elementos tienen la misma prioridad, se extrae primero el que fue insertado antes. Es decir, la cola con prioridad es estable.

Decisión de diseño (`Prioritario`): verMax() y extraerMax() devuelven un objeto Prioritario<T> que encapsula el par (elemento, prioridad). Esto es necesario porque los métodos de utilización 10 y 12 requieren conocer tanto el elemento como su prioridad, y la única vía permitida es la interfaz del TDA. Devolver el par junto mantiene la interfaz reducida a {crear, insertar, extraerMax, verMax, esVacia} sin exponer detalles internos.

3.2 Implementación

Ambas variantes respetan el criterio de desempate FIFO de 3.1.

Variante A - Desordenada

Los elementos se insertan en el primer lugar libre (al final del arreglo), sin mantener ningún orden. extraerMax/verMax recorren el arreglo buscando la prioridad máxima. Para el desempate FIFO, el arreglo se mantiene en orden de inserción y se toma la primera aparición del máximo (la más antigua); en extraerMax se desplazan los elementos restantes a la izquierda para conservar su orden de inserción.

Operación	Complejidad
insertar	O(1)
extraerMax	O(n)
verMax	O(n)

Variante B - Ordenada

El arreglo se mantiene siempre ordenado de forma ascendente por prioridad, de modo que el máximo queda en la última posición ocupada y extraerMax/verMax lo acceden directamente. Para el desempate FIFO, al insertar un elemento con prioridad p se lo ubica antes de los que ya tienen esa misma prioridad; así, el insertado antes queda en el índice más alto del bloque de empate y, como se extrae desde el final, sale primero.

Operación	Complejidad
insertar	O(n)
extraerMax	O(1)
verMax	O(1)

Discusión - ¿una variante es mejor que la otra en todos los casos?

No. Ninguna domina a la otra en todos los casos; la elección depende del contexto de uso. Cada variante hace O(1) una operación a costa de dejar la otra en O(n):

- La desordenada conviene cuando hay muchas inserciones y pocas extracciones/consultas (inserta en O(1)).
- La ordenada conviene cuando hay muchas extracciones/consultas del máximo y relativamente pocas inserciones (extrae y consulta en O(1)).

Si el patrón de uso está equilibrado (por ejemplo, insertar n elementos y luego extraerlos todos), ambas terminan en $O(n^2)$ en total, así que no hay una mejor en el caso general. La decisión se toma según qué operación esté en el camino crítico de la aplicación.

3.3 Utilización

10. combinar(cp1, cp2): ColaPrioridad

Nueva cola con prioridad con todos los elementos de cp1 y cp2, respetando el criterio de desempate de 3.1 (se conserva el orden relativo de inserción entre elementos de igual prioridad). Ambas entradas quedan vacías.

```
public static <T> ColaPrioridad<T> combinar(ColaPrioridad<T> cp1, ColaPrioridad<T> cp2) {
        ColaPrioridad<T> resultado = new ColaPrioridadOrdenada<>();
        while (!cp1.esVacia()) {
        Prioritario<T> e = cp1.extraerMax();
        resultado.insertar(e.getElemento(), e.getPrioridad());
        }
        while (!cp2.esVacia()) {
        Prioritario<T> e = cp2.extraerMax();
        resultado.insertar(e.getElemento(), e.getPrioridad());
        }
        return resultado;
}
```

Se extraen los elementos de cp1 en orden de prioridad (con desempate FIFO) y se insertan en el resultado; luego los de cp2. Como el resultado también aplica desempate FIFO, para una misma prioridad quedan primero los de cp1 (en su orden original) y después los de cp2 (en su orden original). Complejidad temporal: $O((n1 + n2)^2)$ en el peor caso con implementación estática: se realizan O(n1 + n2) operaciones y cada insertar/extraerMax cuesta O(n) en alguna de las variantes.

11. invertirColaConColaPrioridad(c: Cola): Cola

Nueva cola con los elementos de c en orden inverso, usando una **ColaPrioridad** auxiliar.

```
public static <T> Cola<T> invertirColaConColaPrioridad(Cola<T> c) {
        ColaPrioridad<T> cp = new ColaPrioridadOrdenada<>();
        int prioridad = 0;
        while (!c.esVacia()) {
        cp.insertar(c.desencolar(), prioridad);
        prioridad++;
        }
        Cola<T> resultado = new ColaCircular<>();
        while (!cp.esVacia()) {
    resultado.encolar(cp.extraerMax().getElemento());
        }
        return resultado;
}
```

Idea sobre la prioridad: a medida que se recorre c se asigna una prioridad creciente (0, 1, 2, …). Como el último elemento de c recibe la mayor prioridad, extraerMax lo devuelve primero, y el resultado queda invertido. c queda vacía. Complejidad temporal: O(n) operaciones de TDA. Con la variante ordenada, como las prioridades son crecientes, cada insertar cae al final en O(1), por lo que el método resulta O(n).

12. sumarValoresPrioridadPar(cp): entero

Suma de los valores de todos los elementos cuya **prioridad es par**. cp queda con su contenido y orden original al finalizar.

```
public static int sumarValoresPrioridadPar(ColaPrioridad<Integer> cp) {
        ColaPrioridad<Integer> aux = new ColaPrioridadOrdenada<>();
        int suma = 0;
        while (!cp.esVacia()) {
        Prioritario<Integer> e = cp.extraerMax();
        if (e.getPrioridad() % 2 == 0) suma += e.getElemento();
        aux.insertar(e.getElemento(), e.getPrioridad());
        }
        while (!aux.esVacia()) {                    // reconstruir cp
        Prioritario<Integer> e = aux.extraerMax();
        cp.insertar(e.getElemento(), e.getPrioridad());
        }
        return suma;
}
```

Se vacía cp en una cola con prioridad auxiliar (acumulando la suma de los de prioridad par) y luego se reconstruye cp desde la auxiliar. Como el criterio de extracción (prioridad + desempate FIFO) es determinista, reinsertar en el mismo orden de extracción restaura cp de forma idéntica. Complejidad temporal: $O(n^2)$ en el peor caso con implementación estática.

Resumen de complejidades de utilización

#	Método	Complejidad
6	pasarCola	O(n)
7	invertirColaConPila	O(n)
8	invertirColaSinPila	O(n)
9	finalCoincide	O(n1 + n2)
10	combinar	$O((n1+n2)^2)$
11	invertirColaConColaPrioridad	O(n)
12	sumarValoresPrioridadPar	$O(n^2)$

(En 10 y 12 el término cuadrático proviene del costo O(n) de las operaciones de la Cola con Prioridad estática; contando solo operaciones de TDA, ambos realizan O(n) llamadas.)